IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya arts science,commerce and management

Project – High Level Design

on

Cloud cost intelligence platform for retail

Student Name:

Abhinav R Nambiar

Industry Mentor:

Guided by: Ms. Yuktha

Table of Contents

1. Introduction. 6
   1.1. Scope of the document. 6
   1.2. Intended Audience 6
   1.3. System overview. 6

2. System Design. 6
   2.1. Application Design 6
   2.2. Process Flow. 6
   2.3. Information Flow. 7
   2.4. Components Design 8
   2.5. Key Design Considerations 8
   2.6. API Catalogue. 8

3. Data Design. 8
   3.1. Data Model 8
   3.2. Data Access Mechanism 9
   3.3. Data Retention Policies 9
   3.4. Data Migration 9

4. Interfaces 9

5. State and Session Management 10

6. Caching 10

7. Non-Functional Requirements 10
   7.1. Security Aspects 10
   7.2. Performance Aspects 10

8. References

1. Introduction

1.1. Scope of the Document

This document explains the overall design of the **Cloud Cost Calculator project** in simple terms. It describes the main components of the system, how they interact, and how data flows from input to output.

1.2. Intended Audience

This document is intended for:

- Project reviewers
- Faculty / guides
- Developers
- Students

1.3. System Overview

The Cloud Cost Calculator is an application that helps users analyze and estimate cloud service costs.

It calculates cost based on:

- Compute usage (hours)
- Storage usage (GB)
- API requests

The system provides:

- Total cost
- Average cost
- Highest cost service
- Graphical visualization

The project can be used in:

- Jupyter Notebook (analysis)
- **Python script (main.py)**

2. System Design

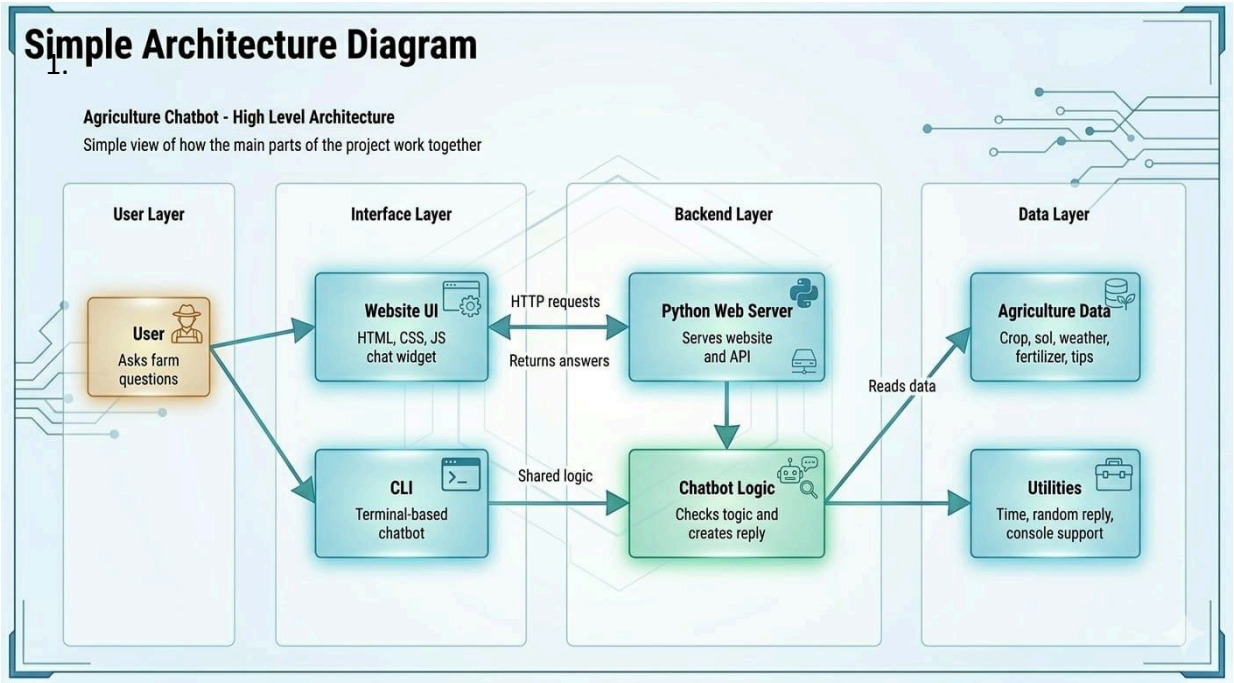
2.1 Application Design

The system consists of the following components:

1. **User Input (Excel File)**
   User provides cloud usage data in Excel format.
2. **Processing Layer (Python)**
   Python processes data and performs cost calculations.
3. **Data Analysis (Pandas)**
   Pandas handles data manipulation and calculations.
4. **Visualization (Matplotlib / Streamlit)**
   Graphs and dashboard display insights.

Simple Architecture Diagram

1.

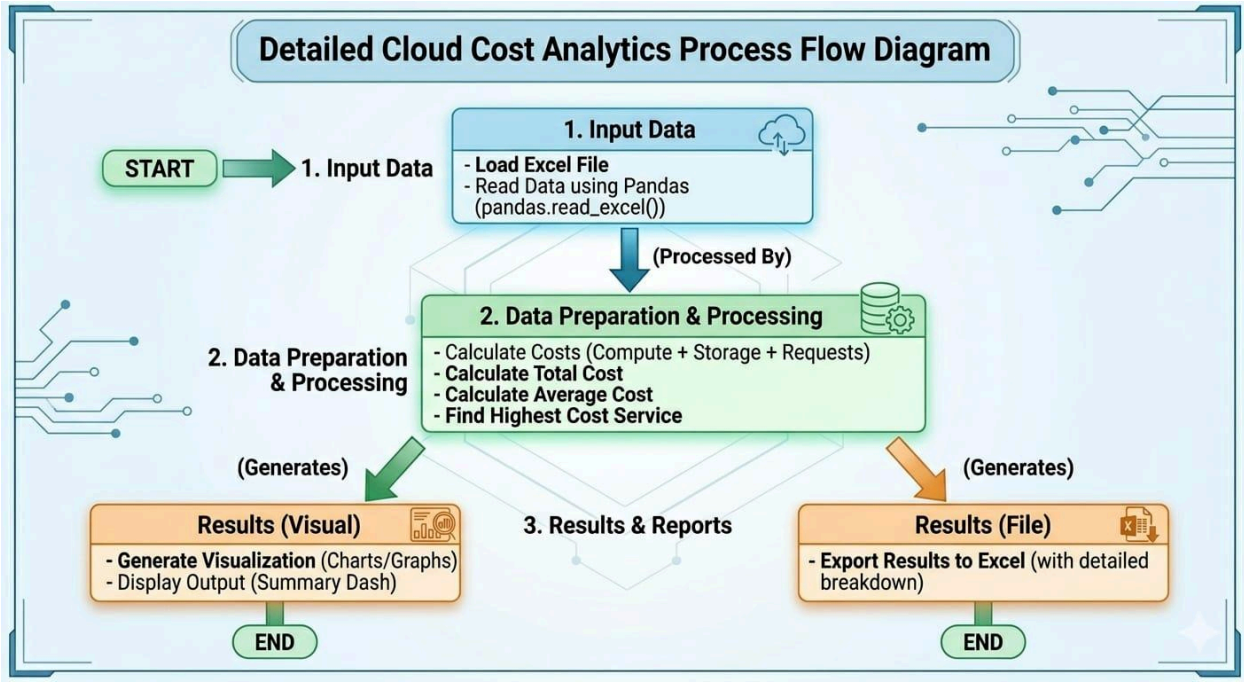


2.2 Process Flow

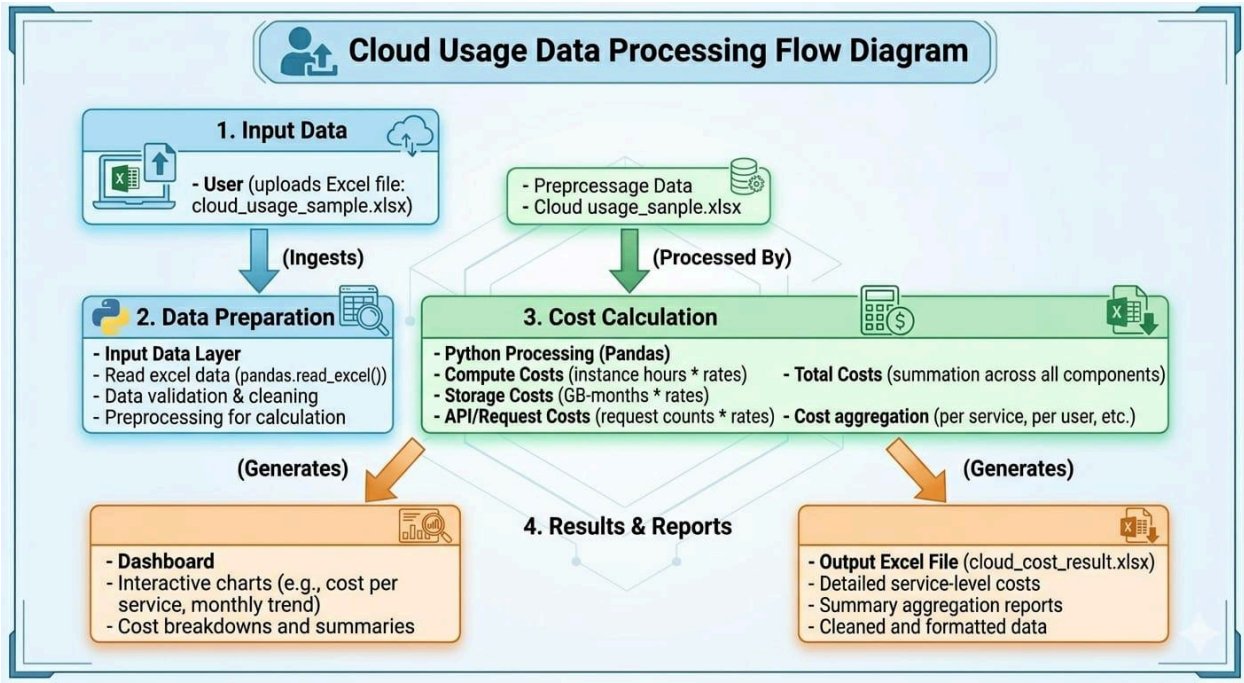
The system works in the following steps:

1. User uploads or provides Excel file
2. Data is read using Pandas
3. Cost is calculated for each service
4. Total and average costs are computed
5. Highest cost service is identified
6. Data is visualized using charts
7. Results are exported to Excel

Process Flow Diagram



2.3 Information Flow Diagram



2.4 Components Design

Component	Purpose	File
Notebook	Data analysis	retail_cloud_cost_calculator.ipynb
Script	Main execution	main.py
Dashboard	UI visualization	dashboard.py

	Purpose	File
Dataset	Input data	cloud_usage_sample.xlsx
Output File	Result storage	cloud_cost_result.xlsx
Documentation	Project details	README.md

2.5 Key Design Considerations

- Simple and easy to use
- No database required
- Uses local files (Excel)
- Fast processing using Pandas
- Easily extendable for more services
- Suitable for academic and demo purposes

2.6 API Catalogue

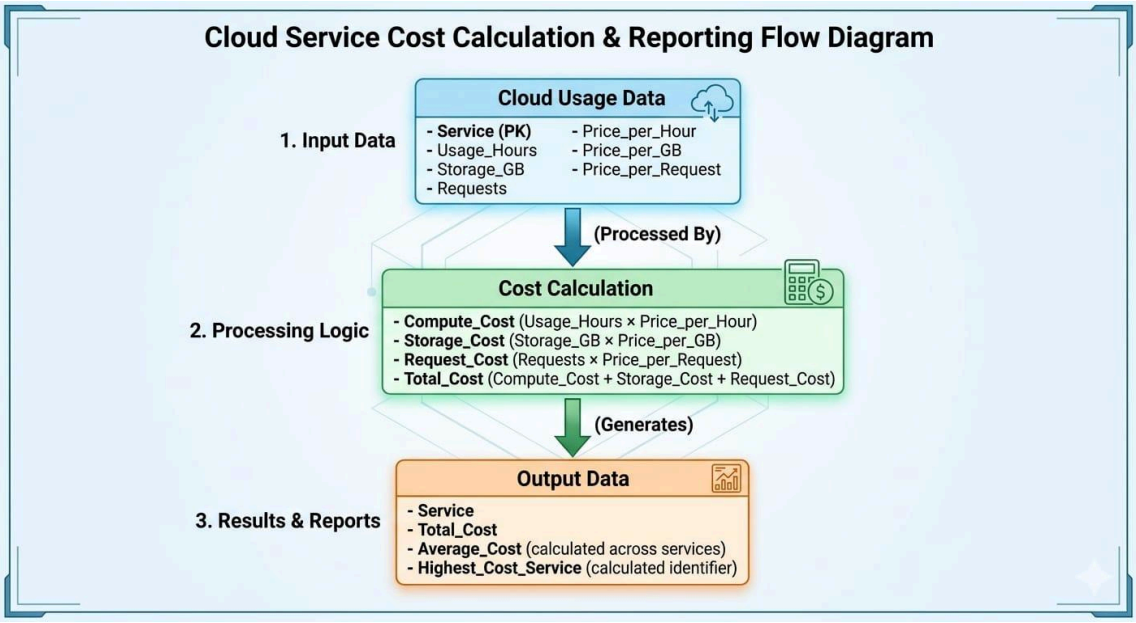
API / Function	Use
`read_excel()`	Load input data
`calculate_cost()`	Compute costs
`to_excel()`	Export results

3. Data Design

3.1 Data Model

Field	Description
Service	Cloud service name
Usage_Hours	Compute usage
Storage_GB	Storage usage
Requests	API usage
Price_per_Hour	Cost per hour
Price_per_GB	Cost per GB
Price_per_Request	Cost per request

Data Model Diagram



3.2 Data Access Mechanism

- Data is read using Pandas:

pd.read_excel()

- Stored in DataFrame for processing

3.3 Data Retention Policies

- No long-term storage
- Data is stored locally
- Output is saved as Excel file
- No user history stored

3.4 Data Migration

Currently:

- Manual Excel upload

Future improvements:

- Database integration
- Cloud storage
- API-based data input

4. Interfaces

The system provides:

- **Jupyter Notebook Interface** → for analysis
- **Python Script Interface** → for execution
- **Dashboard Interface** → interactive UI using Streamlit

5. State and Session Management

- System is stateless
- Each execution is independent
- No session tracking
- Dashboard session exists only during runtime

6. Caching

- No caching implemented
- Data is processed in memory
- Suitable for small to medium datasets

7. Non-Functional Requirements

7.1 Security Aspects

- Local execution ensures data privacy
- No external data sharing
- No authentication required
- Safe for academic usage

7.2 Performance Aspects

- Fast computation using Pandas
- Efficient for moderate datasets
- Lightweight system
- Real-time dashboard updates

8. References

Python Documentation

Pandas Documentation

Matplotlib Documentation

Streamlit Documentation

Project Files:

- `main.py`
- `dashboard.py`
- `retail_cloud_cost_calculator.ipynb`
- `README.md`